

Report conversation

Why does `random.uniform` allow the lower number at top or bottom but `random.randint` throws an exception?

The behavior of `random.uniform` and `random.randint` in Python's `random` module is defined by their respective implementations and intended use cases.

1. `random.uniform(a, b)` : This function returns a random floating-point number N such that $a \leq N \leq b$ or $b \leq N \leq a$. The key point here is that `random.uniform` is designed to work with floating-point numbers, where the order of a and b does not matter. It's more flexible and is often used in simulations or modeling where such flexibility is beneficial. If a is greater than b , it simply swaps the values internally.
2. `random.randint(a, b)` : This function returns a random integer N such that $a \leq N \leq b$. However, it requires that a is less than or equal to b . If a is greater than b , it raises a `ValueError`. The reason for this stricter requirement is likely rooted in the traditional use of ranges for integers, where the start of the range is less than or equal to the end of the range. This is consistent with how ranges are generally handled in Python (consider the `range(a, b)` function, which also requires $a \leq b$).

In summary, the difference in behavior is due to the different use cases and design decisions behind these functions. `random.uniform` is designed for flexibility with floating-point numbers, while `random.randint` adheres to the more traditional and strict interpretation of integer ranges.